

SJob Program

El programa `sjob` es un programa en **C** para:

- Copiar periódicamente archivos de una ruta a otra
- Mover periódicamente archivos de una ruta a otra
- Remover periódicamente archivos de una ruta
- Ejecutar periódicamente comandos con archivos encontrados en una ruta

Generalidades

Para su ejecución el programa debe ejecutarse `sjob /archivo/de/configuración` y el mismo trabajará según el archivo de configuración, mostrando más o menos mensajes (con marcas de tiempo) dependiendo de si la configuración es en modo depuración o no. Como el programa está pensado para ser usado como un servicio, y en la mayoría de los Linux modernos se ejecutan los servicios bajo SystemD y SystemD reporta incluyendo marcas de tiempo, bajo ese sistema se usa `sjob -s /archivo/de/configuración`, lo que hace que el programa no incluya mensajes con marcas de tiempo (pero SystemD sí).

El programa `sjob` :

- Lee el archivo de configuración dado como primer parámetro
- En el archivo de configuración puede haber opciones individuales no numeradas `DEBUG` , `DRY_RUN` , `LOOP` , `START_HOUR` y `TIMEFMT` que se explicarán.
- En el archivo de configuración pueden haber opciones numeradas `COPY_JOBx` (con x un entero), `MOVE_JOBx` , `REMOVE_JOBx` y `CMD_JOBx` .
 - Las opciones `COPY_JOBx` deben tener sus parámetros conexos con el mismo identificador numérico. Se usan para que el programa vaya a una ruta, busque unos archivos y los copie a otra ruta
 - Las opciones `MOVE_JOBx` deben tener también sus parámetros conexos con el mismo identificador numérico. Se usan, como el nombre lo indica para que el programa vaya a una ruta, busque unos archivos y los mueva a otra ruta.
 - Las opciones `REMOVE_JOBx` de igual manera tienen parámetros conexos. Se usan para que el programa vaya a una ruta, busque archivos y los remueva (los borre del sistema de archivos).
 - Las opciones `CMD_JOBx` también tienen parámetros conexos. Se usa para que el programa vaya a una ruta, busque archivos y ejecute un comando dando como parámetro los archivos encontrados.

Opciones base

DEBUG

La opción `DEBUG` se usa para que el programa emita numerosos mensajes, reportando cuando comienza cada job y cuando termina, y cuántos archivos está procesando en cada job. La opción tiene la sintaxis `DEBUG=booleano` donde booleano puede ser `True` , `Yes` , `1` , (sin importar mayúsculas) o `False` , `No` , o `0` (sin importar mayúsculas).

Este parámetro es **opcional**.

DRY_RUN

La opción `DRY_RUN` se usa para simular, es decir, que el programa buscará en las rutas los archivos según las condiciones pero no "hará", es decir, no copiará, moverá, removerá o ejecutará comandos.

Este parámetro es claramente **opcional**.

TIMEFMT

La opción `TIMEFMT` se usa para indicar o especificar cómo se muestran los avisos de mensajes. Si el programa recibe la opción `-s` (o `--systemd` equivalente) entonces la ignora porque no muestra mensajes marcados con el tiempo. Si no se usa el programa usa `%Y-%m-%d %H:%M:%S` para marcar los mensajes (o año, mes, día separados por guiones y hora, minuto y segundo separados por dos puntos).

Este parámetro es **opcional**.

LOOP

Cuánto tiempo debe esperar el programa entre ejecución y ejecución. Recibe un **número entero** (que interpreta como número de segundos), o un número seguido de `m` (indica minutos), `h` (horas) o `d` (días). El programa procesa todos las tareas configuradas y espera el tiempo indicado para volver a iniciar desde la tarea con menor número. Nótese que el programa tiene en cuenta cuánto se toman las tareas, así que si `LOOP=300` y las taras toman 5 segundos esperará 295 (y no 300) segundos para comenzar el ciclo de nuevo. Las unidades `d`, `h`, `m` no son sensibles a las mayúsculas.

NOTA: Esta sentencia era la anterior **DELAY**, pero se cambió para que sea más acorde con el funcionamiento real del programa.

Este parámetro es **opcional**. Si no se da se asume como 0 y el programa ejecuta las tareas una única vez y termina (de manera exitosa).

START_HOUR

A qué horas del día deben comenzar a procesarse los jobs. Reconoce los siguientes formatos:

- H o un número entero solitario. Se interpreta como H:00 , es decir la hora en punto indicada, entre 0 y 23 .
- H:MM o H:M o HH:MM , de 0:00 a 23:59 . Se interpreta en horario militar local.
- HMM o HHMM , de 000 (0:00) a 2359 (23:59). Hora militar local.

Si la hora indicada es anterior al momento actual (si son las 3:00PM o 15:00 y START_HOUR=3:00) entonces el programa esperará a que sea dicha hora el siguiente día para comenzar.

Este parámetro es **opcional**.

Opciones de tarea numeradas

Tareas de copia

Las tareas de copia se definen usando los parámetros COPY_JOBx (título), y los parámetros de dónde, hacia dónde, qué nombres de archivos y filtros de "más nuevo" y "más viejo":

- `COPY_JOBx` Simplemente un título para la tarea. Son visibles en la consola (cuando no está ejecutándose con SystemD) o en el log del sistema (cuando se está ejecutando con SystemD).
- `COPY_SOURCEx` Ruta (entre comillas dobles) de dónde se deben tomar los archivos a copiar. Si se usa SystemD debe ser una ruta absoluta (comenzando con `/`).
- `COPY_TARGETx` Ruta (entre comillas dobles) a dónde se deben copiar los archivos. Igual que con la fuente, si se usa SystemD debe ser una ruta absoluta.
- `COPY_EXPRx` Filtro de nombres de archivos a copiar. Si no se da busca "todos los no ocultos" o `*` . Este parámetro es **opcional**.
- `COPY_NEWERx` Filtro de edad de los archivos a copiar. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro copia los archivos que cumplen `COPY_EXPRx` y (**AND**) sean más nuevos que el tiempo indicado. Este parámetro es **opcional**.
- `COPY_OLDERx` Filtro de edad de los archivos a copiar. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro copia los archivos que cumplen `COPY_EXPRx` y (**AND**) sean más viejos que el tiempo indicado. Este parámetro es **opcional**.
- `COPY_PREx` Comando a ejecutar antes de comenzar la copia de los archivos. Este parámetro es **opcional**. Si se da un `_PREx` el comando debe terminar con éxito (código de retorno `0`) para que se ejecute el resto del `COPY_JOBx` .
- `COPY_POSTx` Comando a ejecutar después de hacer la copia de los archivos. Este parámetro es **opcional**.

Notas: Las tareas de copia sobrescriben los archivos destino que encuentre con el mismo nombre. Los filtros de edad comparan exclusivo, es decir "más nuevo que" y no "más nuevo o igual que" y así. Las tareas intentan mantener el dueño, el grupo y los permisos de los archivos copiados. Se sugiere que los comandos `PREx` y `POSTx` se nombre con ruta completa (`/usr/bin/su` y no `su`).

Tareas de movimiento

Las tareas de movimiento se definen usando los parámetros `MOVE_JOBx` (título), y los parámetros de dónde, hacia dónde, qué nombres de archivos y filtros de "más nuevo" y "más viejo":

- `MOVE_JOBx` Simplemente un título para la tarea. Son visibles en la consola (cuando no está ejecutándose con SystemD) o en el log del sistema (cuando se está ejecutando con SystemD).
- `MOVE_SOURCEx` Ruta (entre comillas dobles) de dónde se deben tomar los archivos a mover. Si se usa SystemD debe ser una ruta absoluta (comenzando con `/`).
- `MOVE_TARGETx` Ruta (entre comillas dobles) a dónde se deben mover los archivos. Igual que con la fuente, si se usa SystemD debe ser una ruta absoluta.
- `MOVE_EXPRx` Filtro de nombres de archivos a mover. Si no se da busca "todos los no ocultos" o `*` . Este parámetro es **opcional**.
- `MOVE_NEWERx` Filtro de edad de los archivos a mover. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro mueve los archivos que cumplen `MOVE_EXPRx` y (AND) sean más nuevos que el tiempo indicado. Este parámetro es **opcional**.
- `MOVE_OLDERx` Filtro de edad de los archivos a mover. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro mueve los archivos que cumplen `MOVE_EXPRx` y (AND) sean más viejos que el tiempo indicado. Este parámetro es **opcional**.
- `MOVE_PREx` Comando a ejecutar antes de comenzar el movimiento de los archivos. Este parámetro es **opcional**. Si se da un `_PREx` el comando debe terminar con éxito (código de retorno `0`) para que se ejecute el resto del `MOVE_JOBx` .
- `MOVE_POSTx` Comando a ejecutar después de hacer el movimiento de los archivos. Este parámetro es **opcional**.

Notas: Las tareas de movimiento sobrescriben los archivos destino que encuentre con el mismo nombre. Los filtros de edad comparan exclusivo, es decir "más nuevo que" y no "más nuevo o igual que" y así. Las tareas intentan mantener el dueño, el grupo y los permisos de los archivos movidos. Se sugiere que los comandos `PREx` y `POSTx` se nombre con ruta completa (`/usr/bin/su` y no `su`).

Tareas de remoción

Las tareas de remoción se definen usando los parámetros `REMOVE_JOBx` (título), y los parámetros de dónde, qué nombres de archivos y filtros de "más nuevo" y "más viejo":

- `REMOVE_JOBx` Simplemente un título para la tarea. Son visibles en la consola (cuando no está ejecutándose con `SystemD`) o en el log del sistema (cuando se está ejecutando con `SystemD`).
- `REMOVE_SOURCEx` Ruta (entre comillas dobles) de dónde se deben tomar los archivos a mover. Si se usa `SystemD` debe ser una ruta absoluta (comenzando con `/`).
- `REMOVE_EXPRx` Filtro de nombres de archivos a remover. Si no se da busca "todos los no ocultos" o `*` . Este parámetro es **opcional**.
- `REMOVE_NEWERx` Filtro de edad de los archivos a remover. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro remueve los archivos que cumplen `REMOVE_EXPRx` y (**AND**) sean más nuevos que el tiempo indicado. Este parámetro es **opcional**.
- `REMOVE_OLDERx` Filtro de edad de los archivos a mover. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro remueve los archivos que cumplen `REMOVE_EXPRx` y (**AND**) sean más viejos que el tiempo indicado. Este parámetro es **opcional**.
- `REMOVE_PREx` Comando a ejecutar antes de comenzar la remoción de los archivos. Este parámetro es **opcional**. Si se da un `_PREx` el comando debe terminar con éxito (código de retorno `0`) para que se ejecute el resto del `MOVE_JOBx` .
- `REMOVE_POSTx` Comando a ejecutar después de hacer la remoción de los archivos. Este parámetro es **opcional**.

Notas: Las tareas de remoción no pasan por un espacio de reciclaje, entonces no hay cómo recuperar los archivos removidos. Los filtros de edad comparan exclusivo, es decir "más nuevo que" y no "más nuevo o igual que" y así. Se sugiere que los comandos `PREx` y `POSTx` se nombre con ruta completa (`/usr/bin/su` y no `su`).

Tareas de ejecución

Las tareas de ejecución se definen usando los parámetros `CMD_JOBx` (título), y los parámetros de dónde, qué nombres de archivos y filtros de "más nuevo" y "más viejo":

- `CMD_JOBx` Simplemente un título para la tarea. Son visibles en la consola (cuando no está ejecutándose con SystemD) o en el log del sistema (cuando se está ejecutando con SystemD).
- `CMD_SOURCEx` Ruta (entre comillas dobles) de dónde se deben tomar los archivos a mover. Si se usa SystemD debe ser una ruta absoluta (comenzando con `/`).
- `CMD_COMMANDx` Comando a ejecutar sobre los archivos encontrados. El comando típicamente incluye `{}` para indicar dónde deberán darse nombres de comandos como parámetros. Si por ejemplo se quisiera copiar por SSH al servidor respaldo y ruta `/backup` se usaría `CMD_COMMAND5="scp {} respaldo:/backup"` (suponiendo que sea la tarea 5). Debe ser claro que este parámetro es **obligatorio**.
- `CMD_EXPRx` Filtro de nombres de archivos a remover. Si no se da busca "todos los no ocultos" o `*` . Este parámetro es **opcional**.
- `CMD_NEWERx` Filtro de edad de los archivos a remover. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro remueve los archivos que cumplen `CMD_EXPRx` y (**AND**) sean más nuevos que el tiempo indicado. Este parámetro es **opcional**.
- `CMD_OLDERx` Filtro de edad de los archivos a mover. Si no se da busca todos. Si se da debe ser un número de segundos (un entero sin sufijo), o un número de minutos (con el sufijo `m`), o número de horas (con `h`), o número de días (con `d`). Con este parámetro remueve los archivos que cumplen `CMD_EXPRx` y (**AND**) sean más viejos que el tiempo indicado. Este parámetro es **opcional**.
- `CMD_REPLACEx` Cadena de texto a reemplazar en `CMD_COMMANDx` por los nombres de los archivos encontrados y filtrados. Si no se usa el programa asume `{}` . Este parámetro es **opcional**.
- `CMD_MULTIPLEx` Este parámetro es booleano. Se usa para indicar que el comando se ejecutará con todos los nombres en una sola ejecución y separados por espacios (cuando es verdadero) o que el comando se ejecutará una vez por cada uno de los nombres de los

archivos encontrados (cuando es falso). Como los otros parámetros booleanos, se puede usar `True` , `Yes` , `1` (sin comillas y sin importar mayúsculas), o también `False` , `No` , o `0` (también sin comillas y sin importar mayúsculas donde aplique). Según el ejemplo anterior, si tanto `archivo1` como `archivo2` cumplen los filtros y se tiene

`CMD_MULTIPLE5=True` entonces se ejecutaría `scp archivo1 archivo2 respaldo:/backup` (una ejecución con todos los archivos); pero si se define `CMD_MULTIPLE5=False` entonces ejecutaría primero `scp archivo1 respaldo:/backup` y luego `scp archivo2 respaldo:/backup` (una ejecución por cada archivo).

Notas: Los comandos es mejor especificarlos con ruta completa (como `/usr/bin/scp`) y no solo el nombre (como `scp`). Los filtros de edad comparan exclusivo, es decir "más nuevo que" y no "más nuevo o igual que" y así.

Ejemplos de archivo de configuración

Ejemplo de archivo de configuración #1

Un archivo muy sencillo sería

```
DELAY=3600
```

```
COPY_JOB1="Copia los logs de /db2/logs a /respaldo/db2/archived"
```

```
COPY_SOURCE1="/db2/logs"
```

```
COPY_TARGET1="/respaldo/db2/archived"
```

El anterior archivo, si se guardase como `/etc/copialogs.conf` se usaría invocando (como `root` o como un usuario que pueda leer de `/db2/logs` y escribir en `/respaldo/db2/archived`) `sjob /etc/copialogs.conf` y no más. El programa corre indefinidamente hasta que se corte con **CONTROL-C** o se cancele con `kill <PID>` .

Ejemplo de archivo de configuración #2

Un archivo un poco más complejo sería

```
DELAY=1d
DEBUG=No
DRY_RUN=No

CMD_JOB1="Envía a servidor alternativo los logs numerados de /db2/logs"
CMD_SOURCE1="/db2/logs"
CMD_COMMAND1="/usr/bin/rsync {} alternativo:/guardado"
# Archivos con tres dígitos en la extensión
CMD_EXPR1="*.[0-9][0-9][0-9]"
# Archivos menores a un día
CMD_NEWER1=1d

REMOVE_JOB2="Remueve los archivos mayores a un día que estén en /db2/logs"
REMOVE_SOURCE2="/db2/logs"
# Remueve los archivos terminados en tres dígitos
REMOVE_EXPR2="*.[0-9][0-9][0-9]"
# Remueve los mayores a un día de modificados
REMOVE_OLDER2=1d
# Baja la instancia antes de remover los archivos
REMOVE_PRE2=/usr/bin/su - db2inst -c "db2stop -force"
# Sube la instancia luego de la remoción
REMOVE_POST2=/usr/bin/su - db2inst -c "db2start"
```

El anterior archivo se usaría de manera semejante al ejemplo de más arriba. Como hay un REMOVE_PRE2 , el comando dado (el /use/bin/su ... force") debe terminar con éxito (código de retorno 0) para que se inicie la tarea de remoción de archivos.

Ejecución como servicio

Si se ejecuta como servicio y el sistema usa SystemD (como RHEL) entonces se necesitan tres cosas:

- El programa en una ruta definida. Se recomienda usar `/usr/local/bin/sjob`.
- Un archivo de configuración. Se recomienda usar `/etc/sjob.conf`.
- Un archivo de definición de servicio. Se recomienda usar `/etc/systemd/system/filemover.service` (o de pronto `sjob.service`). Recuérdese que al agregar o cambiar el archivo de definición de servicio deberá ejecutarse `systemctl daemon-reload`.
- Como cualquier servicio, si el servicio es `filemover.service` entonces para ver el estado (si está en ejecución) se usa `systemctl status filemover`, para iniciar se usa `systemctl start filemover` y para detener se usa `systemctl stop filemover`.
- El programa `sjob` solo lee el archivo de configuración al inicio, así que si se edita el archivo de configuración (i.e. `/etc/sjob.conf`) deberá usarse `systemctl restart filemover`.
- Los logs de ejecución de un servicio se revisan con `journalctl` como cualquier servicio.

Ejemplo de archivo de definición de servicio

Si se quiere usar como servicio (recomendado) se crearía un archivo y se ubicaría por ejemplo como `/etc/systemd/system/filemover.service`:

```
[Unit]
Description=sjob File Mover Service
After=network.target local-fs.target

[Service]
Type=simple
# The command to execute along with the configuration file argument
ExecStart=/usr/local/bin/sjob -s /etc/sjob.conf
# Automatically restart the service if it crashes or is killed
Restart=always
RestartSec=10
# Standard output and error logging goes to the systemd journal
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Nótese que se incluye la opción `-s` para que el programa no emita mensajes con marca de tiempo (ya que SystemD incluye éstas). Nótese que `ExecStart` deberá apuntar a la ruta real donde se puso el programa `sjob` y también con el nombre completo (con ruta) a dónde está la configuración. Luego de almacenado el archivo se usan `systemctl daemon-reload` y `systemctl start filemover`.

Ejemplo de revisión de estado

Teniendo cargado el servicio (sin importar si está detenido) se usa `systemctl status filemover` (si se creó como `filemover.service`):

```

1 e@rhel10:/tmp$ sudo systemctl status filemover
2 o filemover.service - sjob File Mover Service
3     Loaded: loaded (/etc/systemd/system/filemover.service; disabled; p
4     Active: inactive (dead)
5
6 Jun 26 17:51:33 rhel10 sjob[5339]: Files to process (showing up to 5):
7 Jun 26 17:51:33 rhel10 sjob[5339]: Job '#1 ### Copy all from /tmp/uno t
8 Jun 26 17:51:33 rhel10 sjob[5339]: Starting MOVE Job: '#2 ### Move some
9 Jun 26 17:51:33 rhel10 sjob[5339]: Found 11 files matching criteria.
10 Jun 26 17:51:33 rhel10 sjob[5339]: Files to process (showing up to 5):
11 Jun 26 17:51:33 rhel10 sjob[5339]: Job '
#2 ### Move some from /tmp/uno to /tmp/dos' done.
12 Jun 26 17:51:33 rhel10 sjob[5339]: All jobs finished. Time spent: 0.01s
13 Jun 26 17:52:13 rhel10 systemd[1]: Stopping filemover.service - sjob Fi
14 Jun 26 17:52:13 rhel10 systemd[1]: filemover.service: Deactivated succe
15 Jun 26 17:52:13 rhel10 systemd[1]: Stopped filemover.service - sjob Fil
16 e@rhel10:/tmp$

```

Arriba se puede observar que en el momento no está en ejecución (pero sí ejecutó) y los últimos pocos mensajes. El comando muestra marcas de tiempo.

Ejemplo de revisión de registros

Habiendo ejecutado al menos una vez el servicio se puede usar `journalctl` , por ejemplo para ver lo último:

```

1 e@rhel10:/tmp$ sudo journalctl -u filemover -e
2 Jun 26 17:23:25 rhel10 sjob[4940]: Starting job: #2 ### Move some from
3 Jun 26 17:23:25 rhel10 sjob[4940]: Job '#2 ### Move some from /tmp/uno
4 Jun 26 17:23:35 rhel10 systemd[1]: Stopping filemover.service - sjob Fi
5 Jun 26 17:23:35 rhel10 systemd[1]: filemover.service: Deactivated succe
6 Jun 26 17:23:35 rhel10 systemd[1]: Stopped filemover.service - sjob Fil
7 Jun 26 17:50:23 rhel10 systemd[1]: Started filemover.service - sjob Fil
8 Jun 26 17:50:24 rhel10 sjob[5301]: Starting job: #1 ### Copy all from /
9 Jun 26 17:50:24 rhel10 sjob[5301]: Job '#1 ### Copy all from /tmp/uno t
10 Jun 26 17:50:24 rhel10 sjob[5301]: Starting job: #2 ### Move some from
11 Jun 26 17:50:24 rhel10 sjob[5301]: Job '#2 ### Move some from /tmp/uno
12 Jun 26 17:50:37 rhel10 systemd[1]: Stopping filemover.service - sjob Fi
13 Jun 26 17:50:37 rhel10 systemd[1]: filemover.service: Deactivated succe
14 Jun 26 17:50:37 rhel10 systemd[1]: Stopped filemover.service - sjob Fil
15 Jun 26 17:51:33 rhel10 systemd[1]: Started filemover.service - sjob Fil
16 Jun 26 17:51:33 rhel10 sjob[5339]: == INITIALIZING JOB RUNNER ==
17 Jun 26 17:51:33 rhel10 sjob[5339]: DEBUG: True
18 Jun 26 17:51:33 rhel10 sjob[5339]: DELAY: 300 seconds
19 Jun 26 17:51:33 rhel10 sjob[5339]: TIMEFMT: %Y-%m-%d %H:%M:%S
20 Jun 26 17:51:33 rhel10 sjob[5339]: DRY_RUN: False
21 Jun 26 17:51:33 rhel10 sjob[5339]: =====
22 Jun 26 17:51:33 rhel10 sjob[5339]: Starting COPY Job: '#1 ### Copy all
23 Jun 26 17:51:33 rhel10 sjob[5339]: Found 118 files matching criteria.
24 Jun 26 17:51:33 rhel10 sjob[5339]: Files to process (showing up to 5):
25 Jun 26 17:51:33 rhel10 sjob[5339]: Job '#1 ### Copy all from /tmp/uno t
26 Jun 26 17:51:33 rhel10 sjob[5339]: Starting MOVE Job: '#2 ### Move some
27 Jun 26 17:51:33 rhel10 sjob[5339]: Found 11 files matching criteria.
28 Jun 26 17:51:33 rhel10 sjob[5339]: Files to process (showing up to 5):
29 Jun 26 17:51:33 rhel10 sjob[5339]: Job '#2 ### Move some from /tmp/uno
30 Jun 26 17:51:33 rhel10 sjob[5339]: All jobs finished. Time spent: 0.01s
31 Jun 26 17:52:13 rhel10 systemd[1]: Stopping filemover.service - sjob Fi
32 Jun 26 17:52:13 rhel10 systemd[1]: filemover.service: Deactivated succe
33 Jun 26 17:52:13 rhel10 systemd[1]: Stopped filemover.service - sjob Fil

```

Nota: Arriba muestra registros de una ejecución con configuración que incluye `DEBUG=True` por lo que muestra algunos de los nombres de los archivos procesados y numerosos mensajes.